OAT 3: Sobrescrita de Métodos e Polimorfismo em Java

Conceitos fundamentais da Programação Orientada a Objetos

Autor: Mateus Oliveira
Victor Ferreira
Andrei Luis
David Lima

Agenda

O objetivo dessa apresentação é explorar a sobrescrita de métodos, a anotação @Override, ligação dinâmica e polimorfismo prático em Java.

1. Questão 1:
Sobrescrita e anotação @Override

2. Questão 2:
Polimorfismo e Ligação Dinâmica

3. Questão 3:
Reaproveitamento com super

4. Questão 4:
Hierarquia de regras de bônus

5. Questão 5:
Tipos de referência vs. Objeto

6. Conclusão:
Considerações Finais e Referências

Questão 1: Sobrescrita e @Override

- **Anotação `@Override`:**
  - Validação em tempo de compilação.
  - Evita erros de digitação (ex: *formatarhoras*).
- **Sem `@Override`:**
  - Cria um novo método sem subscrever o original.
- **Alterar Retorno (String → void):**
  - Erro de compilação: quebra a assinatura.

> 🛡️ **Regra: O @Override garante a assinatura exata do método ancestral.**

```
class Relogio { public String formatarHora(int h, int m) { return h + ":" + m; } } class RelogioDigital extends Relogio { @Override public String formatarHora(int h, int m) { return String.format("%02d:%02d", h, m); } }
```

Questão 2: Polimorfismo e Ligação Dinâmica

- **Ligação Dinâmica (Dynamic Dispatch):**
  - O compilador só valida a assinatura na classe base (`Notificacao`).
  - A decisão do objeto real (`SMS` ou `Email`) ocorre em **runtime**.

Por que o compilador não sabe antes?
A entrada do `Scanner` é imprevisível. O Java resolve o método com base na instância na memória no momento da execução.

```
Notificacao n; if (opcao == 1) { n = new SMS(); }
else { n = new Email(); } // Chamada polimórfica (Runtime)
n.enviar();
```

Questão 3: Reuso com super e Pilha de Execução

- **Mecanismo do `super.sacar(valor)`:**
  - Reaproveita a lógica original sem duplicar código.

Pilha de Execução:

1. Chamada de `sacar()` na subclasse.
2. Pula para `ContaBancaria.sacar()` via `super`.
3. Subtrai o valor do saldo original.
4. Retorna e subtrai a taxa fixa (R$ 1,00).

```
class ContaComTaxa extends ContaBancaria { @Override public void sacar(double valor) { // 1. Pula para o método da classe pai super.sacar(valor); // 2. Aplica a taxa local this.saldo -= 1.00; } }
```

Questões 4 e 5: Polimorfismo na Prática

unex

Q4: Hierarquia de Bônus

- **Funcionário:** 10% do salário.
- **Gerente:** 20% do salário.
- **Diretor:** 20% + R$ 1.000,00 fixos.
- Método `imprimirBonus(Funcionario f)` aceita qualquer subclasse.

Q5: Referência vs. Objeto Real

- `Funcionario func = new Diretor();`
- **Chamada específica:** `func.agendarReuniao()` falha na compilação porque a referência é `Funcionario`.
- **Método sobrescrito:** `func.calcularBonus()` executa a versão da classe `Diretor` (Objeto real).

Considerações finais

- **Pontos de interesse:**
  - A anotação `@Override` é indispensável para prevenir erros silenciosos durante o desenvolvimento.
  - A ligação dinâmica garante flexibilidade ao código, resolvendo chamadas de métodos conforme os objetos reais criados em tempo de execução.
  - O uso de `super` permite a reutilização inteligente de código em classes derivadas sem duplicação de regras de negócio.
  - O tipo da variável de referência limita o que o compilador enxerga, mas a execução sempre respeita a implementação do objeto real instanciado.

Referências

- **Referências do projeto:**

  - DEITEL, Paul; DEITEL, Harvey. **Java: Como Programar**. 10. ed. São Paulo: Pearson Education do Brasil, 2016.
  - SIERRA, Kathy; BATES, Bert. **Use a Cabeça! Java**. 2. ed. Rio de Janeiro: Alta Books, 2007.
  - ORACLE. **Java Platform, Standard Edition Documentation**. Disponível em: . Acesso em: 2026.